

Chapter Overview**7.1 Arrays**

- 7.1.1 Declaring One-Dimensional Arrays
- 7.1.2 Variations in Array Declaration
- 7.1.3 Accessing Individual Elements of Array
- 7.1.4 Entering Data in Arrays
- 7.1.5 Reading Data from Arrays

7.2 Searching in Arrays

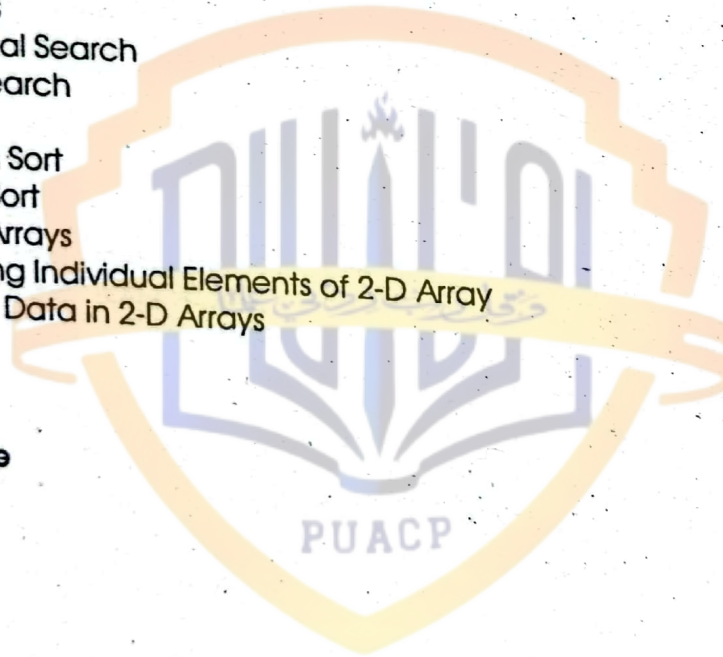
- 7.2.1 Sequential Search
- 7.2.2 Binary Search

7.3 Sorting Arrays

- 7.3.1 Selection Sort
- 7.3.2 Bubble Sort

7.4 Two-Dimensional Arrays

- 7.4.1 Accessing Individual Elements of 2-D Array
- 7.4.2 Entering Data in 2-D Arrays

7.5 Dynamic Arrays**7.6 Control Arrays****Programming Exercise****Short Questions****Multiple Choices****Fill in the Blanks****True/False**

7.1 Arrays

An array is a set of variables that can store similar type of data. This set of variables consists of different memory locations. These locations are adjacent in memory and are indicated with same name. Each memory location can store one value and is called element of array. The elements of the array are accessed with reference to their position or location number. This position number is called index or subscript.

The position of each element in array is different. The first element of array has an index value of 0. The index is incremented by 1 for each next element. If an array has five elements, the index numbers of these elements will be 0, 1, 2, 3 and 4. The index value or subscript is written in parentheses after the name of array.

The basic purpose of using array is to store a large number of similar types of data. For example, hundred variables are required to store the marks of hundred students. The use of individual variables will take a lot of time. It is also very difficult and time-consuming task to perform different processing on these variables. This task can be performed more easily by using arrays. The marks of 100 students can be stored in one array with 100 elements.

Example

Suppose an array **ARR** can store five integer values. In this case, the name of the array is **ARR** and its five elements are **ARR(0)**, **ARR(1)**, **ARR(2)**, **ARR(3)** and **ARR(4)**. The values written in parentheses along with array name indicate subscript values. These values indicate the locations of different array elements in the memory where the values of these elements are stored. It is shown in the following figure:

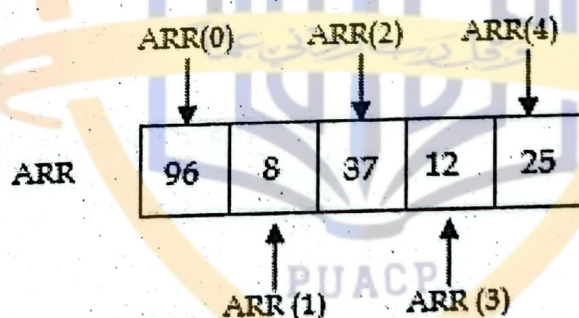


Figure: An array **ARR** with five elements

7.1.1 Advantages / Uses of Arrays

Some advantages of arrays are as follows:

- Arrays can store a large number of values with single name.
- Arrays are used to process many values easily and quickly.
- The values stored in an array can be sorted easily.
- A search process can be applied on arrays easily.

7.1.2 Declaring One-Dimensional Arrays

One-dimensional array is used to store different values of similar type as a single row or column. It is also called linear array, vector array and 1-D array.

Syntax

The syntax of declaring one-dimensional array is as follows:

Dim <array name> (length) **As** <type>

The following line declares an array to store the marks of five students:

`Dim Marks(4) As Integer`

This statement will create an array in the memory that consists of five memory locations numbered from 0 to 4. We can store five integers in this array.



Figure: A one-dimensional array of five elements in memory

Each block in the array is called **cell** or **element**. Each element is assigned a unique number. This number is called **index** or **subscript** of that element. The index is used to access an individual element of the array. Normally, index starts from 0. So, **Marks(0)** refers to the first element of the array, **Marks(1)** refers to the second element and so on.

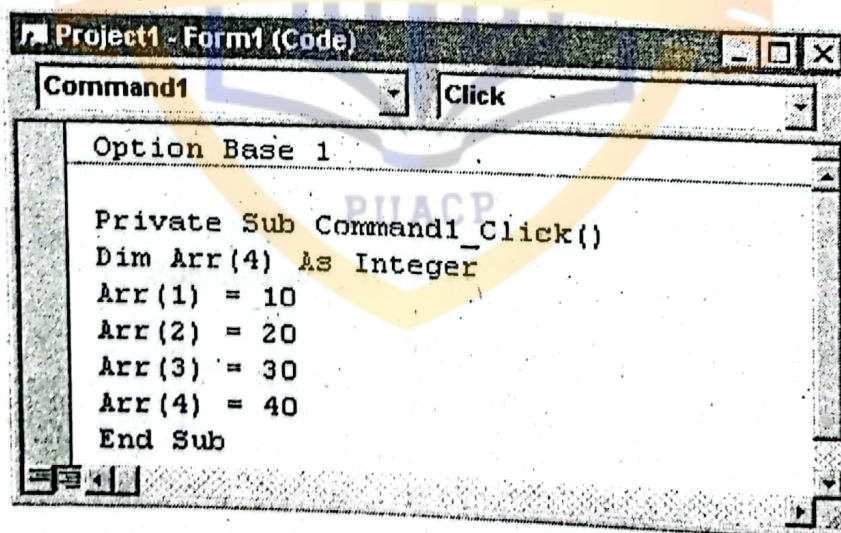
7.1.3 Variations in Array Declaration

By default, the index of an array starts from 0 but the starting index can be changed. It can be started from any number. An example of such array is as follows:

`Dim Marks(1 To 5) As Integer`

The above array can also hold five elements but the index of first element is 1 and the index of last element is 5.

The statement **Option Base 1** is used in the generation declarations section to change the default option of array index. If this option is used, the index of first element of array will be 1 instead of 0.

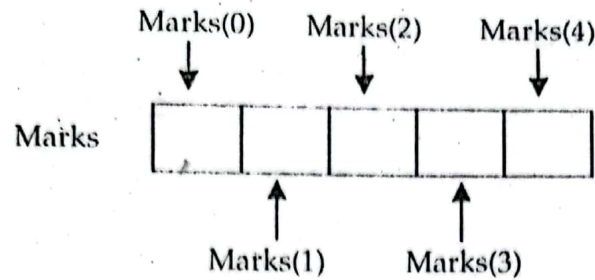


The above example uses **Option Base 1** in general declarations section. So, the length of the array is 4 not 5.

7.1.4 Accessing Individual Elements of Array

The array name and index of the element is used to access an individual element of an array. The following statement will store 100 in the first element of **Marks** array.

`Marks(0) = 100`



The value of index can be given as constant or variable to access an individual element of array. The following example uses a variable as an index:

$N = 2$

Marks(N) = 100

The above line stores 100 in third element of the array. The variable N is used as index.

7.1.5 Entering Data in Arrays

Data can be entered in arrays by using the name of array and index of the element. The following statements enter data in all elements of the array Marks:

Marks(0) = 94

Marks(1) = 37

Marks(2) = 59

Marks(3) = 76

Marks(4) = 92

The above method of entering data in array becomes very lengthy and time-consuming if the array is very large. For example, 100 statements are required to store data in an array with 100 elements. This process can be made very simple and short by using loops. Loops are used to access the individual elements of array with less number of statements.

Program 7.1

Write a program that uses For...Next loop to get five inputs from the user and stores it in an array.

1. Start a new Standard EXE project and place a button on the form.
2. Double click the button and add the following code in its Click event:

The screenshot shows a code editor window titled 'Project1 - Form1 (Code)'. It contains the following code:

```

Command1 Click
Private Sub Command1_Click()
    Dim Marks(4) As Integer
    For i = 0 To 4
        Marks(i) = InputBox("Enter marks:", "Marks", "0")
    Next
End Sub
  
```

3. Run the project and click the button. The input box will appear for five times to get five inputs from the user.

How It Works?

The above example uses the counter variable of For...Next loop as index. The value of counter variable i is 0 in first iteration. The statement Marks(i) refers to the first element of the array. In second iteration, the value of i is 1, so Marks(i) refers to the second element of array and so on. In each iteration, input of the user is stored in different element of the array.

7.1.6 Reading Data from Arrays

The data stored in array can be accessed by accessing an individual element of array. The following statement displays the value third element of array:

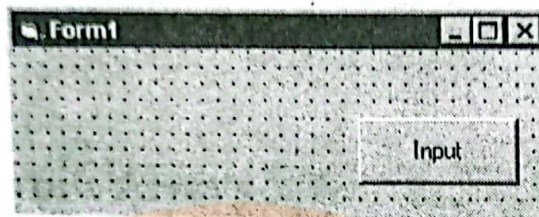
Print Marks(2)

The above method of accessing data in array becomes very lengthy and time-consuming if array is very large. This can also be made very simple and short by using loops.

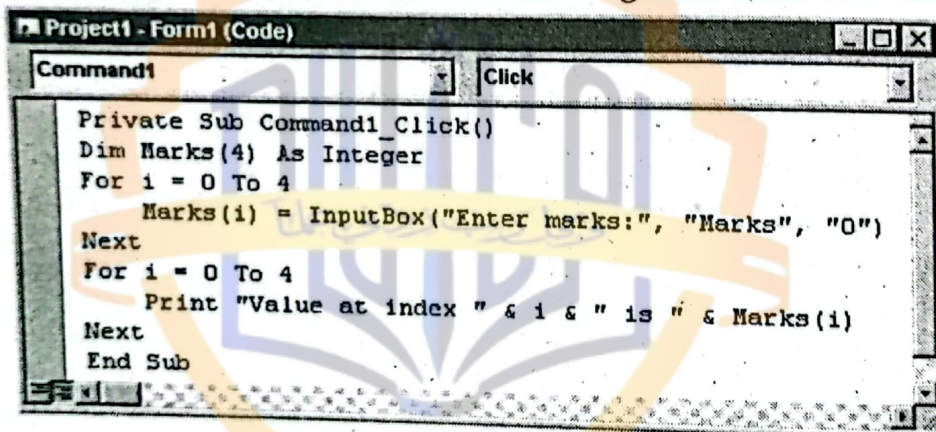
Program 7.2

Write a program that uses two For...Next loops. The first loop gets five inputs from the user and stores it in an array. The second loop prints the values stored in that array.

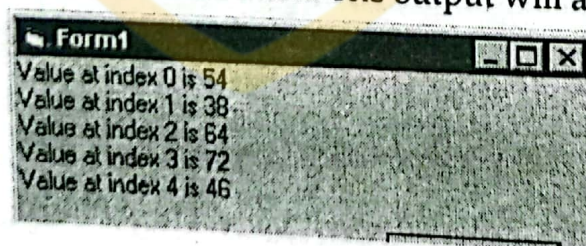
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:



4. Run the project and click the button. The output will appear as follows:



How It Works?

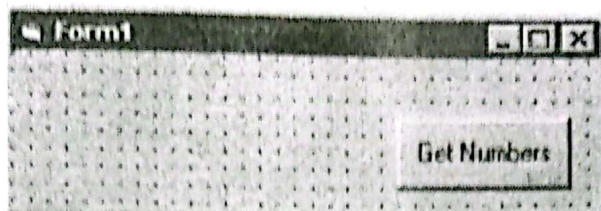
The second loop of above example uses the counter variable *i* as index to display array values. In first iteration, the value of *i* is 0, so Marks(*i*) in the Print statement will display the value of first element of array. In second iteration, the value of *i* is 1, so Marks(*i*) refers to the second element of array and so on.

Program 7.3

Write a program that gets five numbers from the user and stores them in the array. It then displays the sum and average of all numbers in the array.

1. Start a new Standard EXE project.

2. Design the form as follows:



3. Double click the button and add the following code in its Click event:

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim Num(4) As Integer, Sum As Integer
    Sum = 0
    For i = 0 To 4
        Num(i) = InputBox("Enter a number:", "Number", "0")
    Next
    For i = 0 To 4
        Sum = Sum + Num(i)
    Next
    Print "The sum is " & Sum
    Print "The average is " & Sum / 5
End Sub

```

4. Run the project and click the button. The input box will appear five times.
 5. Enter any five numbers. The result will appear as follows:



How It Works?

The above example initializes the variable **Sum** to 0. The first loop gets five inputs. The second loop again visits all elements of the array. In first iteration, the value of counter variable is 0. So it adds the value of first elements and the value of **Sum** and stores the result in **Sum**. In second iteration, it adds the value of second element and **Sum** and then stores the result back to **Sum** and so on. Suppose the user enters the following values in the array.

3	7	5	9	6
---	---	---	---	---

The execution of the second loop to get the sum of all values will work as follows:

Iteration No.	Value of i	Statement Executed	Value of Sum
1 st	0	Sum = Sum + Num(0)	3
2 nd	1	Sum = Sum + Num(1)	10
3 rd	2	Sum = Sum + Num(2)	15
4 th	3	Sum = Sum + Num(3)	24
5 th	4	Sum = Sum + Num(4)	30

When the loop terminates, the value of **Sum** is 30. In the end, the sum and average is displayed on the form.

Program 7.4

Write a program that gets five numbers from the user and stores them in the array. It then displays the numbers in reverse order.

1. Start a new Standard EXE project.
2. Design the form as follows:



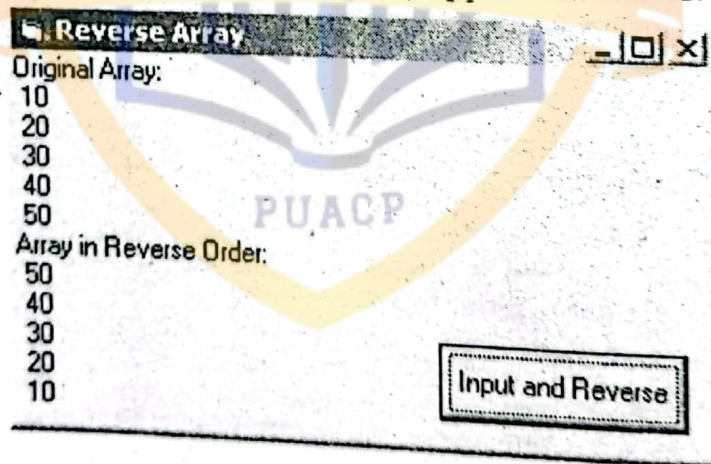
3. Double click the button and add the following code in its Click event:

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1 Click()
    Dim arr(4) As Integer
    For i = 0 To 4
        arr(i) = InputBox("Enter number:")
    Next
    Print "Original Array:"
    For i = 0 To 4
        Print arr(i)
    Next
    Print "Array in Reverse Order:"
    For i = 4 To 0 Step -1
        Print arr(i)
    Next
End Sub

```

4. Run the project and click the button. The input box will appear five times.
5. Enter any five numbers. The result will appear as follows:

**Program 7.5**

Write a program that reads n values in a list and prints the product of the values and the position at which it resides in the list i.e. $\text{product} = L(1) \times 1$, $\text{product} = L(2) \times 2$ and so on.

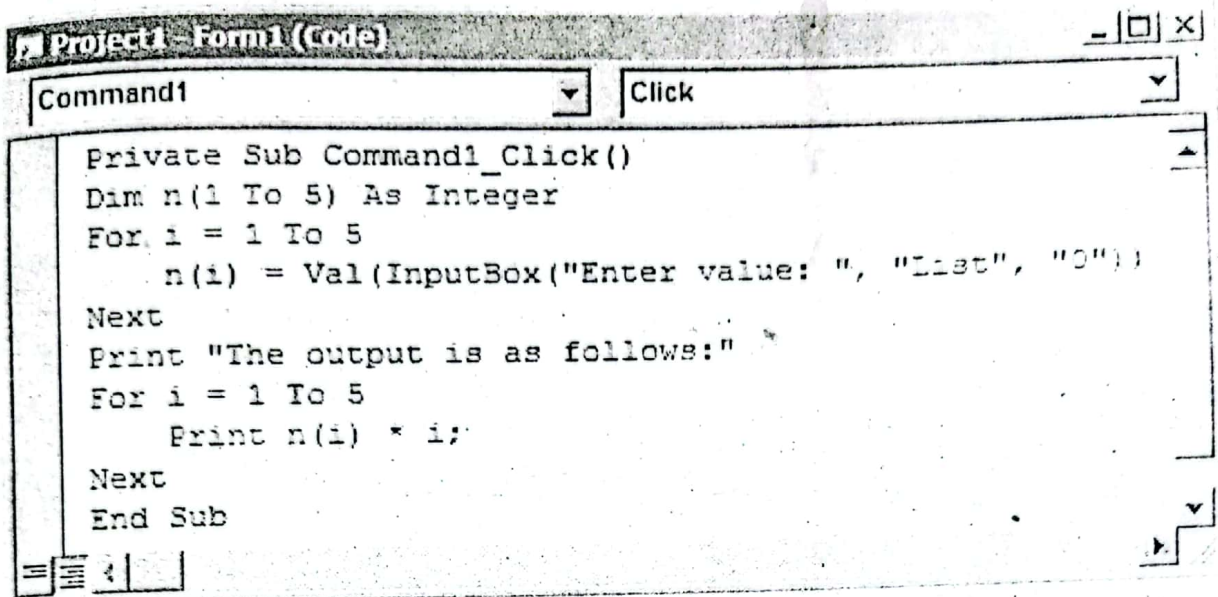
Note: The list does not change.

Example

Position	1	2	3	4	5
Value	5	3	7	1	4

Output: 5 6 21 4 20

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event:

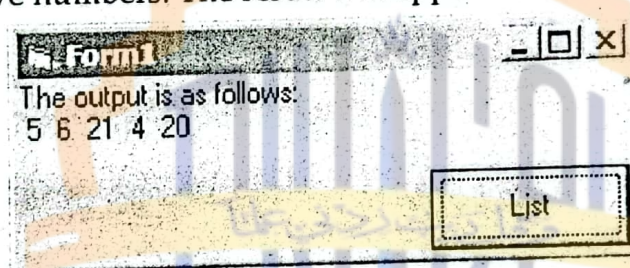


```

Private Sub Command1_Click()
    Dim n(1 To 5) As Integer
    For i = 1 To 5
        n(i) = Val(InputBox("Enter value: ", "List", "0"))
    Next
    Print "The output is as follows:"
    For i = 1 To 5
        Print n(i) * i
    Next
End Sub

```

4. Run the project and click the button. The input box will appear five times:
5. Enter any five numbers. The result will appear as follows:



7.2 Searching in Arrays

Searching is a process of finding the required data in the array. Searching becomes more important when the length of the array is very large.

7.2.1 Sequential Search

Sequential search is also called linear search or serial search. It is a simple way to search an array for the desired value. It follows the following steps to search a value in array:

1. Visit the first element of the array and compare its value with the required value.
2. If the value of array matches with the desired value, the search is complete.
3. If the value of array does not match, move to the next element and repeat the same process.

Loops are frequently used to visit the elements of array for searching a value. You can start the counter variable of loop from 0 and move it to the last index of the array. For example, if the array to be searched has 10 elements, you will start the counter variable from 0 and move to 9.

Program 7.6

Write a program that stores five values in an integer array. It gets a number from the user and finds it in the array using sequential search.

1. Start a new Standard EXE project and place a button on the form.

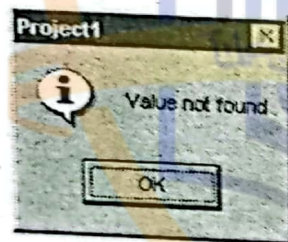
2. Double click the button and add the following code in its Click event:

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim Num(5) As Integer, d As Integer, id As Integer
    id = -1
    Num(0) = 10
    Num(1) = 20
    Num(2) = 30
    Num(3) = 40
    Num(4) = 50
    d = InputBox("Enter number to find:", "Searching", "0")
    For i = 0 To 4
        If Num(i) = d Then
            id = i
            Exit For
        End If
    Next
    If id = -1 Then
        MsgBox "Value not found", vbInformation
    Else
        MsgBox "Value is at index " & id, vbInformation
    End If
End Sub

```

3. Run the project and click the button. An input box will appear.
 4. Enter any number to find in the array. If the given number is present in the array, its index will be displayed, otherwise a message will appear to inform that the number is not found in the array as follows:



Unsuccessful search



Successful search

How It Works?

In the above example, the array is filled with five values i.e. 10, 20, 30, 40 and 50. It gets a number from the user to find in d. The For...Next loop visits all elements of array.

- First of all, the value of i is 0 so Num(i) refers to the first element of the array. The value of first element is compared with the value of d. If both values are similar, it indicates that the search is successful and the value of index is stored in the variable id and the control exits from the loop.
- If the value of d and Num(i) are not similar, the control moves to the start of the loop. The value of i becomes 1.
- This process is repeated until the value is found or the loop terminates.

7.2.2 Binary Search

Binary search is a quicker method of searching for value in the array. It takes less time than sequential search.

- It locates the middle element of array and compares with the search number.
- If they are equal, search is successful and the index of middle element is returned.

- If they are not equal, reduces the search to half of the array.
- If the search number is less than the middle element, the first half of the array search, otherwise the second half of the array is search.
- The process continues until the number is found or loop completes without successful search.

Binary search very quick but it can search an array only if it is sorted. It cannot be applied on an unsorted array.

Program 7.1

Write a program that stores five values in an integer array. It gets a number from the user and finds it in the array using binary search.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:

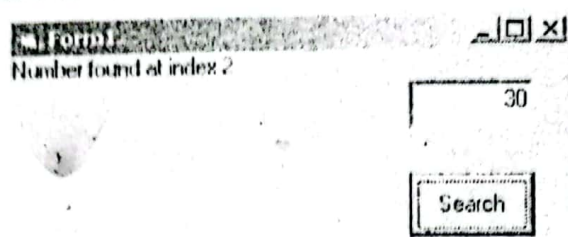
```

Private Sub cmdSearch_Click()
    Form1.Cls
    Dim arr(4) As Integer, num As Integer
    Dim m As Integer, s As Integer, e As Integer
    arr(0) = 10
    arr(1) = 20
    arr(2) = 30
    arr(3) = 40
    arr(4) = 50
    s = 0
    e = 4
    num = Text1.Text
    While (s <= e)
        m = (s + e) / 2
        If (num = arr(m)) Then
            Print ("Number found at index " & m)
            Exit Sub
        ElseIf num < arr(m) Then
            e = m - 1
        Else
            s = m + 1
        End If
    Wend
    Print ("Number not found.")
End Sub

```

4. Run the project.

5. Enter any number in textbox and click Search button. If the given number is present in the array, its index will be displayed, otherwise a message will appear to inform that the number is not found in the array as follows:



7.3 Sorting Arrays

Sorting is a process of arranging the values of array in a particular order. An array can be sorted in two orders:

1. Ascending Order

In ascending order, the smallest value is stored in the first element of array; second smallest value is stored in the second element and so on. The largest value is stored in the last element. Following figure shows an array sorted in ascending order.

12	25	33	37	48	57	86	92
----	----	----	----	----	----	----	----

Figure: Sorted array in ascending order

2. Descending Sort

In descending order, the largest value is stored in the first element of array; second largest value is stored in the second element and so on. The smallest value is stored in the last element. Following figure shows an array sorted in ascending order.

92	86	57	48	37	33	25	12
----	----	----	----	----	----	----	----

Figure: Sorted array in descending order

7.3.1 Selection Sort

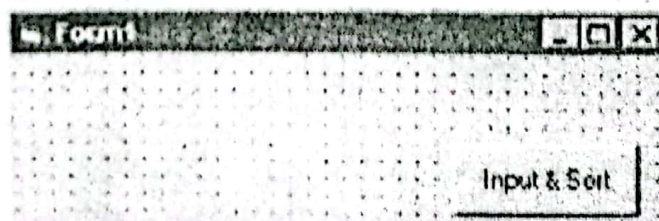
Selection sort is a technique that sorts an array. It selects an element in the array and moves it to its proper position. Selection sort works as follows:

1. Find the minimum value in the list
2. Swap it with the value in the first position
3. Sort the remainder of the list excluding the first value

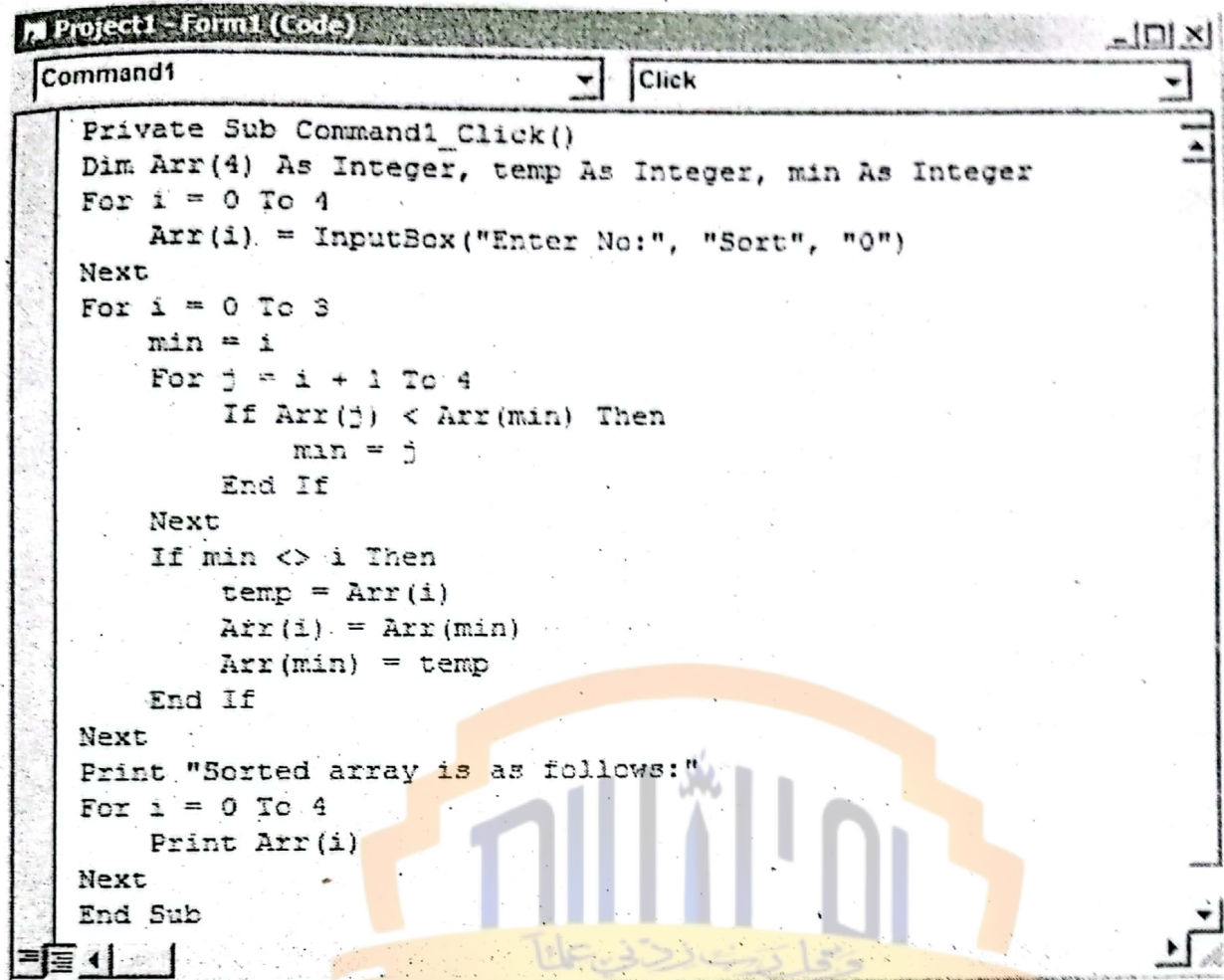
Program 7.8

Write a program that gets five inputs from the user in an array and then sorts this array in ascending order.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:

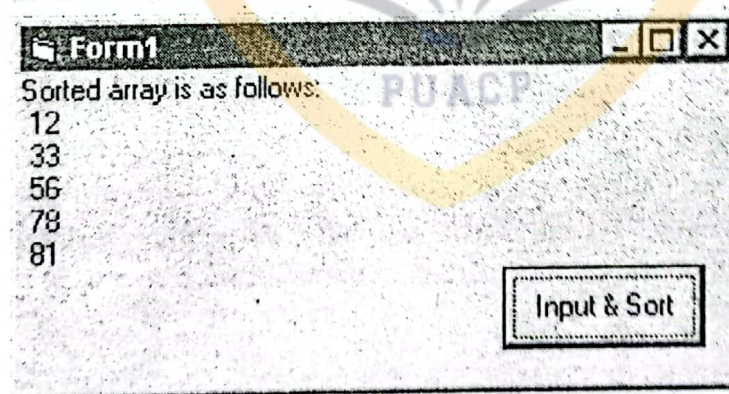


```

Private Sub Command1_Click()
    Dim Arr(4) As Integer, temp As Integer, min As Integer
    For i = 0 To 4
        Arr(i) = InputBox("Enter No:", "Sort", "0")
    Next
    For i = 0 To 3
        min = i
        For j = i + 1 To 4
            If Arr(j) < Arr(min) Then
                min = j
            End If
        Next
        If min <> i Then
            temp = Arr(i)
            Arr(i) = Arr(min)
            Arr(min) = temp
        End If
    Next
    Print "Sorted array is as follows:"
    For i = 0 To 4
        Print Arr(i)
    Next
End Sub

```

4. Run the project and click the button. The input box will appear for five times.
 5. Enter any five numbers. The sorted values will be displayed as follows:



How It Works?

Suppose the user enters 56, 78, 33, 81, and 12 in the array. The above example sorts these values as follows:

56	78	33	81	12
----	----	----	----	----

Figure: The unsorted array

This technique of sorting is known as **selection sort**. In the above example, nested loops are used to sort the array. The outer loop moves from 0 to 3 and with each iteration of outer loop, the inner loop moves from $i + 1$ to 4.

The value of i is stored in variable **min** in each iteration. The value in index **min** is compared with all remaining elements. If any value is found less than the value in index **min**, its index is stored in the variable **min**. The value at index i is interchanged with the value at index **min** at the end of each pass if required. The sorting process will work as follows:

Pass 1

Iteration 1

56	78	33	81	12
----	----	----	----	----

$i = 0$ so the value of **min** is also 0. $j = 1$ so 56 is compared with 78. As 78 is not less than 56, there is no change in the value of **min**.

Iteration 2

56	78	33	81	12
----	----	----	----	----

min = 0 and $j = 2$ so 56 is compared with 33. As 33 is less than 56, the value of j is stored in variable **min** so **min** = 2.

Iteration 3

56	78	33	81	12
----	----	----	----	----

min = 2 and $j = 3$ so 33 is compared with 81. As 81 is not less than 33, there is no change in the value of **min**.

Iteration 4

56	78	33	81	12
----	----	----	----	----

min = 2 and $j = 4$ so 33 is compared with 12. As 12 is less than 33, the value of j is stored in variable **min** so **min** = 4.

At this point, the inner loop is completed. The value of **min** has changed in the inner loop so the value at index i and the value at index **min** are interchanged. The position of smallest values is now finalized and the array is as follows:

12	78	33	81	56
----	----	----	----	----

Pass 2

Iteration 1

12	78	33	81	56
----	----	----	----	----

$i = 1$ so the value of **min** is also 1. $j = 2$ so 78 is compared with 33. As 33 is less than 78, the value of j is stored in variable **min** so **min** = 2.

Iteration 2

12	78	33	81	56
----	----	----	----	----

$\text{min} = 2$ and $j = 3$ so 33 is compared with 81. As 81 is not less than 33, there is no change in the value of min .

Iteration 3

12	78	33	81	56
----	----	----	----	----

$\text{min} = 2$ and $j = 4$ so 33 is compared with 56. As 56 is not less than 33, there is no change in the value of min .

At this point, the inner loop is completed. The value of min has changed in the inner loop so the value at index i and value at index min are interchanged. The array is as follows:

12	33	78	81	56
----	----	----	----	----

Pass 3

Iteration 1

12	33	78	81	56
----	----	----	----	----

$i = 2$ so the value of min is also 2. $j = 3$ so 78 is compared with 81. As 81 is not less than 78, there is no change in the value of min .

Iteration 2

12	33	78	81	56
----	----	----	----	----

$\text{min} = 2$ and $j = 4$ so 78 is compared with 56. As 56 is less than 78, the value of j is stored in variable min so $\text{min} = 4$.

At this point, the inner loop is completed. The value of min has changed in the inner loop so the value at index i and value at index min are interchanged. The array is as follows:

12	33	56	81	78
----	----	----	----	----

Pass 4

Iteration 1

12	33	56	81	78
----	----	----	----	----

$i = 3$ so the value of min is also 3. $j = 4$ so 81 is compared with 78. As 78 is less than 81, the value of j is stored in variable min and $\text{min} = 4$.

At this point, the inner loop is completed. The value of min has changed in the inner loop so the value at index i and value at index min are interchanged. The array is as follows:

12	33	56	78	81
----	----	----	----	----

The outer loop has also completed at this point and the program has sorted the array in ascending order.

LBound and UBound Functions

LBound and UBound functions indicate to the starting and ending indexes of an array respectively. These functions can be used if the user is not sure about the starting index and ending index.

The following example will display the values of all elements of an array:

```
For i = LBound(Arr) To UBound(Arr)
    Print Arr(i)
Next
```

7.3.2 Bubble Sort

Bubble sort is also known as exchange sort. It repeatedly visits the array and compares two items at a time. It swaps these two items if they are in the wrong order. It continues to visit the array until no swaps are needed that means the array is sorted.

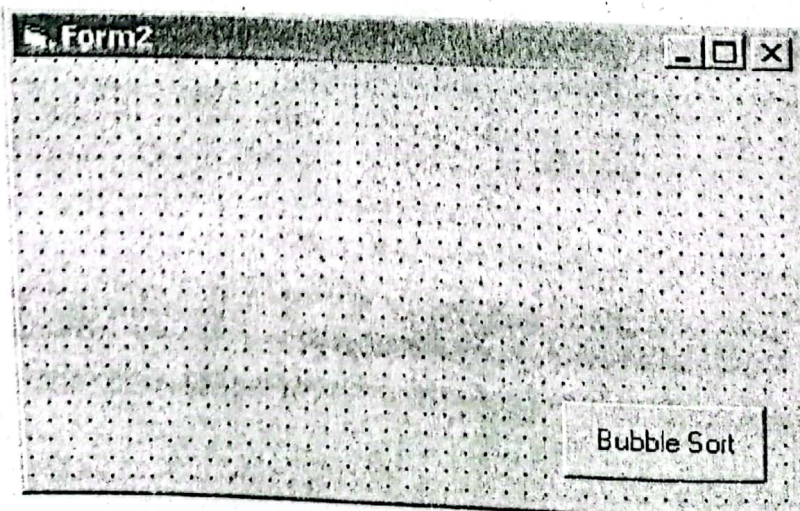
Bubble sort works as follows:

1. Compare adjacent elements. If the first is greater than the second, swap them.
2. Repeat this for each pair of adjacent elements, starting with the first two and ending with the last two. At this point the last element should be the greatest.
3. Repeat the steps for all elements except the last one.
4. Keep repeating for one fewer element each time until there are no more pairs to compare.

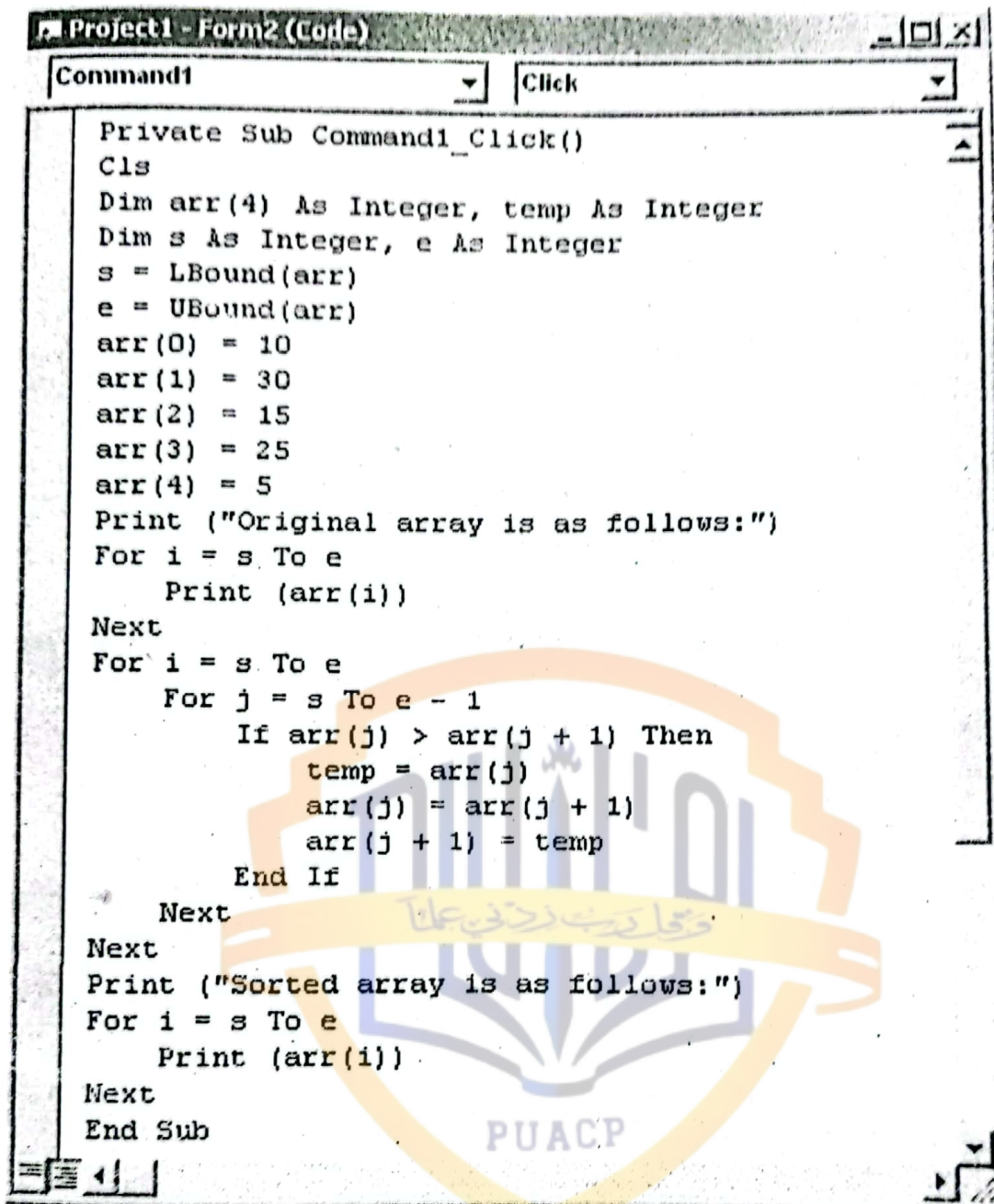
Program 7.9

Write a program that stores five values in an array. It sorts the array using bubble sort. It also displays the values of unsorted and sorted array.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:

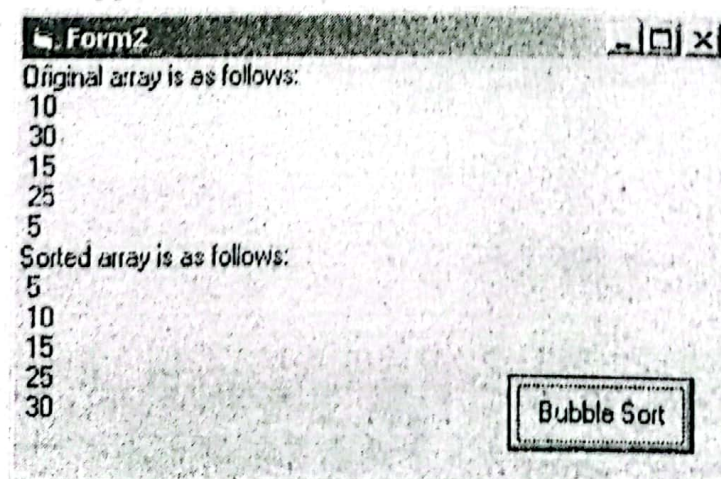


```

Project1 - Form2 (Code)
Command1 Click
Private Sub Command1_Click()
    Cls
    Dim arr(4) As Integer, temp As Integer
    Dim s As Integer, e As Integer
    s = LBound(arr)
    e = UBound(arr)
    arr(0) = 10
    arr(1) = 30
    arr(2) = 15
    arr(3) = 25
    arr(4) = 5
    Print ("Original array is as follows:")
    For i = s To e
        Print (arr(i))
    Next
    For i = s To e
        For j = s To e - 1
            If arr(j) > arr(j + 1) Then
                temp = arr(j)
                arr(j) = arr(j + 1)
                arr(j + 1) = temp
            End If
        Next
    Next
    Print ("Sorted array is as follows:")
    For i = s To e
        Print (arr(i))
    Next
End Sub

```

4. Run the project and click the button.
 5. The form will appear as follows:



Form2

Original array is as follows:

10
30
15
25
5

Sorted array is as follows:

5
10
15
25
30

Bubble Sort

How it Works?

The initial values stored in the array are as follows:

10	30	15	25	5
----	----	----	----	---

Figure: The unsorted array

In the above example, nested loops are used to sort the array. The outer loop moves from 0 to 4 and with each iteration of outer loop, inner loop moves from 0 to $e - (i+1)$.

First of all, the value of i is 0 so the focus of outer loop is on the first element of the array. The sorting process will work as follows:

Pass 1

Iteration 1

10	30	15	25	5
----	----	----	----	---

$j = 0$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 10 with 30. As 10 is not greater than 30, there will be no change in the array.

Iteration 2

10	30	15	25	5
----	----	----	----	---

$j = 1$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 30 with 15. As 30 is greater than 15, both values will be interchanged and the array will be as follows:

10	15	30	25	5
----	----	----	----	---

Iteration 3

10	15	30	25	5
----	----	----	----	---

$j = 2$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 30 with 25. As 30 is greater than 25, both values will be interchanged and the array will be as follows:

10	15	25	30	5
----	----	----	----	---

Iteration 4

10	15	25	30	5
----	----	----	----	---

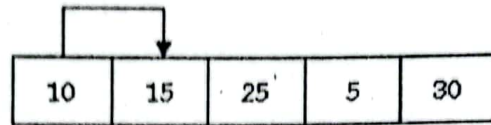
$j = 3$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 30 with 5. As 30 is greater than 5, both values will be interchanged and the array will be as follows:

10	15	25	5	30
----	----	----	---	----

At this point, the inner loop is completed and the largest value in array has moved in the last element. It means that the position of largest value is now finalized. Now the control moves to the beginning of outer loop and the value of i becomes 1.

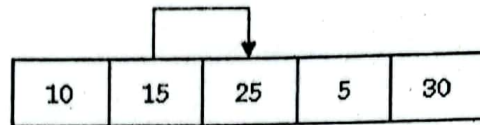
Pass 2

Iteration 1



$j = 0$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 10 with 15. As 10 is not greater than 15, there will be no change in the array.

Iteration 2



$j = 1$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 15 with 25. As 15 is not greater than 25, there will be no change in the array.

Iteration 3



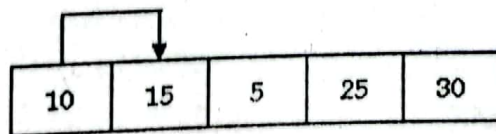
$j = 2$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 25 with 5. As 25 is greater than 5, both values will be interchanged and the array will be as follows:



At this point, the inner loop is completed and the second largest value in array has moved in the second last element. It means that the position of second largest values is now finalized. Now the control moves to the beginning of outer loop and the value of i becomes 2.

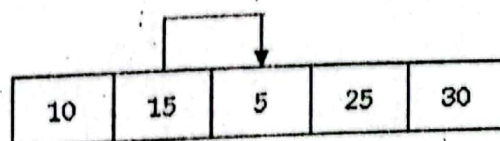
Pass 3

Iteration 1



$j = 0$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 10 with 15. As 10 is not greater than 15, there will be no change in the array.

Iteration 2



$j = 1$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 15 with 5. As 15 is greater than 5, both values will be interchanged.

The array will be as follows:

10	5	15	25	30
----	---	----	----	----

At this point, the inner loop is completed and the third largest value in the array has moved in the third last element. It means that the position of third largest value is now finalized. Now the control moves to the beginning of outer loop and the value of i becomes 3.

Pass 4

Iteration 1

10	5	15	25	30
----	---	----	----	----

$j = 0$ so the statement $\text{arr}[j] > \text{arr}[j+1]$ compares 10 with 5. As 10 is greater than 5, both values will be interchanged and the array will be as follows:

5	10	15	25	30
---	----	----	----	----

At this point, the inner loop is completed and the fourth largest value in the array has moved in the fourth last element. It means that the position of fourth smallest value is now finalized. The position of smallest number is also automatically finalized. Now the outer loop also terminates and the array is sorted in ascending order.

7.4 Two-Dimensional Arrays

Two-dimensional array is also called table, matrix or 2-D array. Two-dimensional array consists of multiple rows and columns. Each element of this array is accessed using two index values. The first index indicates the row and the second index indicates the column.

Syntax

The syntax for declaring 2-D array is as follows:

Dim <array name> (rows, cols) As Type

array_name It indicates the name of array.
rows It indicates the number of rows in the array.
cols It indicates the number of columns in the array.
Type It indicates the type of data to be stored in the array.

Example

DIM table(3, 2) As Integer

The above example declares a two-dimensional array **table**. It contains four rows and three columns. The index of first element of array is 0,0 and the index of last element is 3,2. The following array will be created in the memory:

	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2
2	2,0	2,1	2,2
3	3,0	3,1	3,2

7.4.1 Accessing Individual Elements of 2-D Array

The array name and indexes of row and column are used to access an individual element of 2-D array. For example, the following statement will store 100 in the second column of first row in the array:

```
Arr(0,1) = 100
```

The index for row or column of 2-D array can be given using variables. The following example will work similar to the line above:

```
R = 0
C = 1
Arr(R, C) = 100
```

7.4.2 Entering Data in 2-D Arrays

You can enter data in any element of the array by using the name of array and index of the element. For example, the following statements enter data in the first row of a 2-D array:

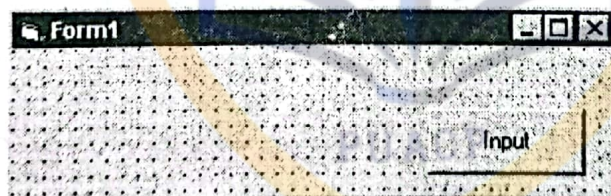
Arr(0, 0) = 10	10 stored in first column of first row
Arr(0, 1) = 20	20 stored in second column of first row
Arr(0, 2) = 30	30 stored in third column of first row
Arr(1, 0) = 40	40 stored in first column of second row
Arr(1, 1) = 50	50 stored in second column of second row
Arr(1, 2) = 60	60 stored in third column of second row

Loops are used to enter data in large 2-D array by using nested loops.

Program 7.10

Write a program that uses nested For...Next loop to store data in a 2-D array.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:

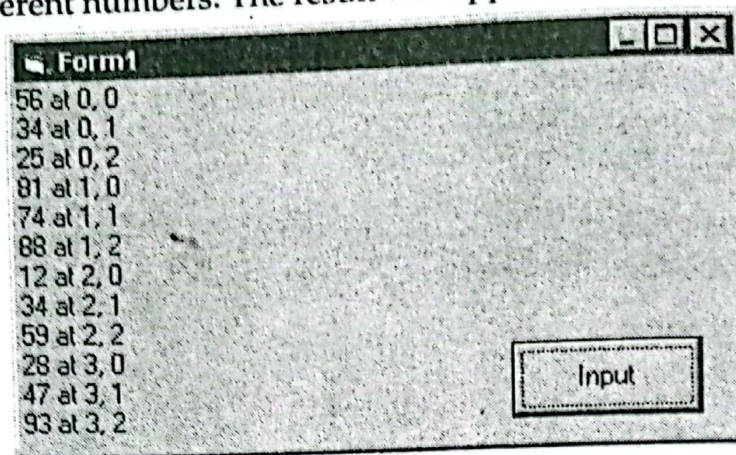
```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    Dim Arr(3, 2) As Integer
    For i = 0 To 3
        For j = 0 To 2
            Arr(i, j) = InputBox("Enter data:", , "0")
        Next
    Next

    For i = 0 To 3
        For j = 0 To 2
            Print Arr(i, j) & " at " & i & ", " & j
        Next
    Next
End Sub

```

4. Run the project and click the button. The input box will appear for 12 times.
5. Enter different numbers. The result will appear as follows:



How It Works?

The above example uses nested loop to access the elements of array. It uses the counter variables of these loops as index. In first iteration of outer loop, the value of *i* is 0. When the control moves to inner loop, the value of *j* is also 0. So it refers to the first column of first row. The value of *i* changes from 0 to 3. The value of *j* changes from 0 to 2 with each iteration of outer loop. In this way, all elements of the array are accessed.

7.5 Dynamic Arrays

Dynamic means changeable. Dynamic array is a type of array in which the number of elements can be added or deleted during program execution. This is done by using **ReDim** statement.

Syntax

In order to use dynamic array, it is declared with empty parentheses as follows.

```
Dim Arr() As Integer
ReDim Arr(5) As Integer
```

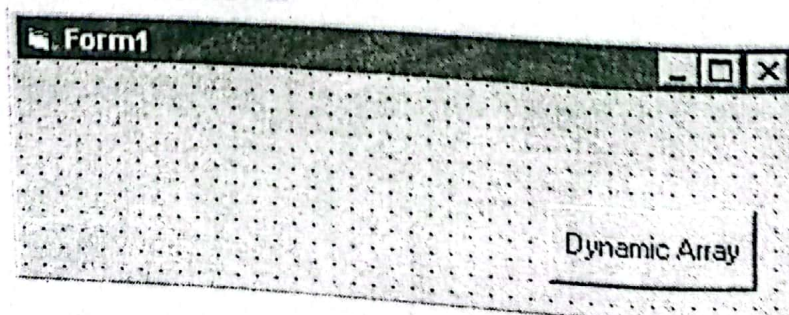
Dynamic arrays are normally used when the length of the array is not known at design time. Dynamic arrays can be declared according to the requirement of the user at run time. When **ReDim** statement is used, any values previously stored in array are deleted. The **Preserve** statement is used along with **ReDim** to preserve previous values as follows:

```
ReDim Preserve Arr(5) As Integer
```

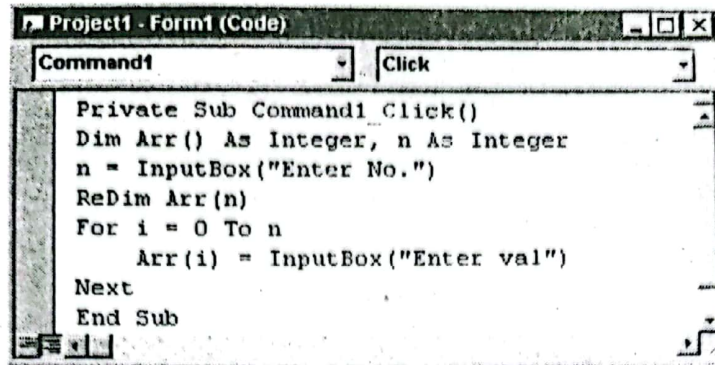
Program 7.11

Write a program that inputs a number from the user and declares an array of integer according to the number given by user. It then inputs values from the user to fill the array.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:



4. Run the project and click the button. An input box will appear.
5. Enter any number and click OK.
6. For...Next loop will get n number of inputs and store them in the array. You can use these values for any purpose.

How It Works?

The above example gets a number from the user in n. The **ReDim** statement creates an array with n elements. It means that the length of array is decided during execution. If the user enters 10 in n, an array of 10 elements will be created. For...Next loop will also execute n number of times to fill the array.

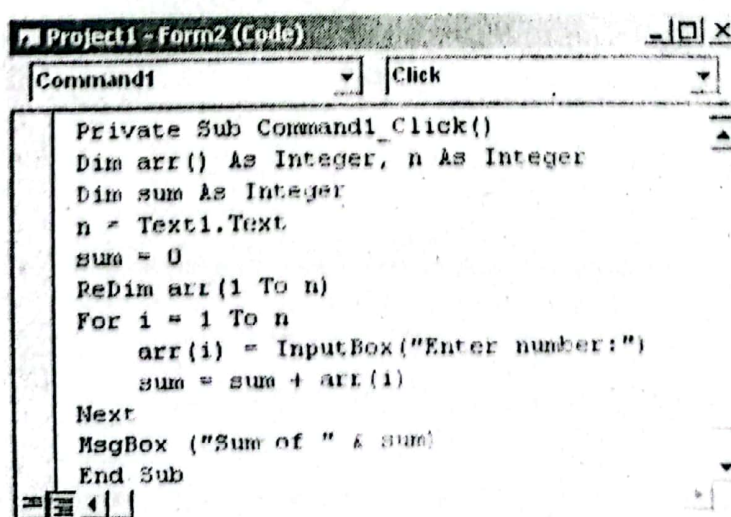
Program 7.12

Write a program that inputs a number from the user and declares an array of integer according to the number given by user. It then inputs values from the user and stores them in the array. It then displays the sum of all numbers.

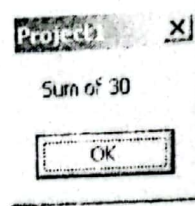
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code in its Click event:



4. Run the project.
5. Enter 3 in textbox and click **Input** button.
6. **For...Next** loop will get *n* number of inputs and store them in the array. The following result will appear:



7.6 Control Arrays

Simple array is a group of consecutive memory locations with same name and type. Similarly, control array is a group of similar controls. For example, a control array can be an array of buttons, labels or textboxes etc. In case of control array, the name of all controls is same. Each control in the control array is identified with the help of its index.

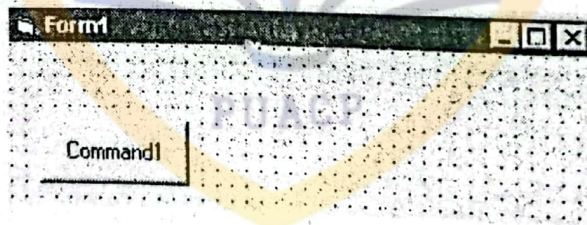
A control array can be created as follows:

1. Place any control on the form.
2. Copy that control.
3. Paste the control on the form. Visual Basic will display a message box to inform that a control with same name already exists. The message will ask whether you want to create a control array? You can click **Yes** to create a control array.

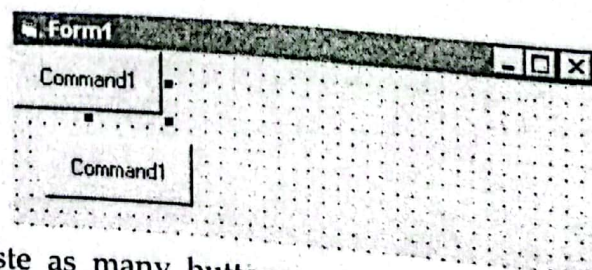
Program 7.13

Write a program that creates a control array of three buttons. The index of button should appear in message box if the user clicks a button.

1. Start a new Standard EXE form.
2. Place a button on it as follows:

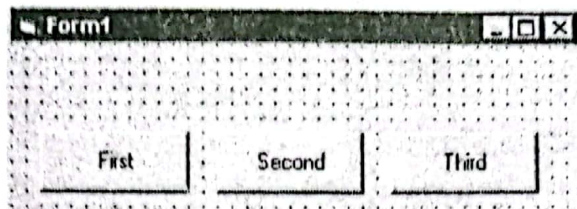


3. Right click on the button and select **Copy** from the menu.
4. Right click again on form and select **Paste** from menu. A message will appear asking "You already have a control name 'Command1'. Do you want to create a control array?"
5. Click **Yes**. A duplicate copy of button will appear on form with same caption and name.

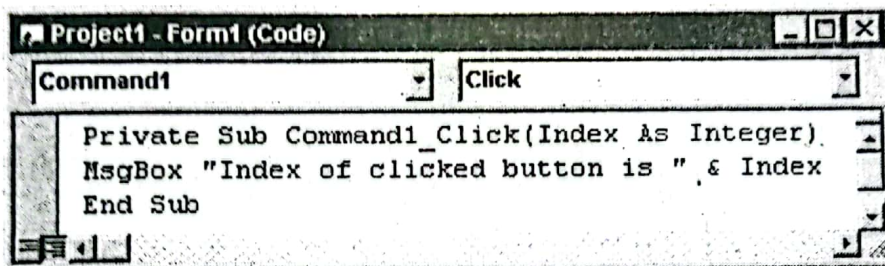


6. You can paste as many buttons as required. Visual Basic will add all these buttons in control array by assigning a unique index to each of them.

7. Paste three buttons and change their captions as follows:



8. The names of these button in the properties window will now be **Command1(0)**, **Command1(1)** and **Command1(2)** respectively.
9. Double click any button and add the following code in its Click event:



10. Run the project and click any of three buttons. A message box will appear showing the index of the clicked button.

How It Works?

The above example uses a control array of three buttons with 0, 1 and 2 index. The difference between individual buttons and array of buttons is that when the user double clicks any button in the array, same block of code appears in the code window. It means that only one block of code is attached to all controls in a control array. The parameter **Index** contains the index of the button that is clicked. The message box displays this index.

Program 7.14

Write a program that inputs names of five students in a control array of textboxes. It inputs marks of the students in another control array of textboxes. It displays average marks, name and marks of the top student in textboxes when the user clicks a button.

1. Start a new Standard EXE form.
2. Design the form as follows:

Student Names	Marks
Average	
Top Student	
Highest Marks	
Process	

Note: The form contains two control arrays of textboxes. The five textboxes below **Student Names** is one array and five textboxes below **Marks** is another array. The remaining textboxes are single textboxes.

3. Add the following code in Click event of Process button:

```

StudentGrades - frmStudentGrades (Code)
cmdAverage Click
Private Sub cmdAverage_Click()
    Dim i As Integer, Max As Integer
    Dim strMax As String
    Dim Total As Integer
    Total = 0
    Max = txtGrades(0).Text
    strMax = txtStudentName(0).Text
    For i = 0 To 4
        If (txtGrades(i).Text > Max) Then
            Max = txtGrades(i).Text
            strMax = txtStudentName(i).Text
        End If
        Total = Total + txtGrades(i).Text
    Next
    txtAverage.Text = Total / 5
    txtHighestName.Text = strMax
    txtHighestGrade.Text = Max
End Sub

```

4. Run the project.
5. Enter values in the textboxes and click **Process** button. The result will appear as follows:

Student Names	Marks
Salman	82
Ahmed	77
Usman	98
Anwar	59
Naeem	33
Average	
Top Student	
Highest Marks	
Process	

Student Names	Marks
Salman	82
Ahmed	77
Usman	98
Anwar	59
Naeem	33
Average	
69.8	
Top Student	
Usman	
Highest Marks	
98	
Process	

Programming Exercise

1. Write a program that get five names from the user and stores them in an array of strings. The program then displays only those names that start with "A".
2. Write a program that gets ten numbers from the users in array and displays the contents of the array in reverse order.
3. Write a program that gets ten numbers from the users in array and displays the maximum number.
4. Write a program that gets ten numbers from the users in array and displays the minimum number.
5. Write a program that gets five numbers from the users in array and displays total number of even and odd numbers stored in the array.
6. Write a program that stores five names in an array of strings. It gets a name from the user and displays whether the name is present in the array or not.

Important Note: The solutions of all questions are available at the end of the book.

Short Questions

Q.1. Define array, element and subscript.

An array is a series of individual variables referenced by same name. The individual variables in an array are referred to as elements. The subscript is the position of the element within the array.

Q.2. How does an array differ from simple variable?

An array can store multiple values but a simple variable can store only one value. Different types of processing can be applied on array easily. However, it is difficult to process multiple values using simple variables.

Q.3. How are individual array elements identified?

A subscript variable is used to identify individual elements in the array. The value fo subscript is written in parentheses with array name.

Q.4. How subscript variables are written?

A subscript variable is written in parentheses with array name.

Q.5. List three uses of array.

1. Array is used to store many values with same name.
2. Array is used to process many values quickly and easily.
3. Array is used to sort many values easily.

Q.6. Distinguish between 1-D array and 2-D array.

1-D array consists of only single row or single column. Each element of this array can be accessed using one index value. 2-D array consists of multiple rows and columns. Each element of this array is accessed using two index values.

Q.7. Explain array manipulation.

The process of performing different operations on array is called array manipulation. Different operations can be performed using array such as finding an element, comparing two arrays, finding the minimum or maximum value in array and sorting an array etc.

Q.8. What is control array?

A control array is a group of controls with the same name. Each control in an array must be of same class. An advantage of using a control array is that the controls share one Click event.

Q.9. Can the user add Controls in a control arrays at the run time.

Controls in a control array can be added and removed at a runtime using 'Load' and 'Unload' statements.

Multiple Choice

1. Variable that holds a large group of similar type of data is called:
 - a. Constant
 - b. Scalar value
 - c. Array
 - d. Multiple variable
2. The elements in an array may be used:
 - a. In accumulators
 - b. in counts
 - c. As reference fields for lookup in statements for printing
 - d. All
3. Each element of array has its own:
 - a. Array.
 - b. Upper bound
 - c. Index
 - d. All
4. An index can be of the data type:
 - a. Currency
 - b. Integer
 - c. String
 - d. All
5. A control array can be created in:
 - a. Design mode
 - b. View mode
 - c. Debug window
 - d. in Print mode
6. In a control array, the subscript of the first element is:
 - a. 0
 - b. 1
 - c. Count - 1
 - d. Count
7. A set of controls with same name whose individual elements are identified with subscript is called:
 - a. Control group
 - b. Control array
 - c. Collection array
 - d. group collection
8. If the index value is larger than the upper bound then:
 - a. An out-of-bounds error will occur
 - b. A division by zero error will occur.
 - c. Both a and b
 - d. None
9. Which of the following are properties of a control array?
 - a. Count, Type and Name
 - b. Count, Item, Name, Index
 - c. Count, Item, LBound, and Ubound
 - d. Index, Size, Name
10. When adding a control to a control array during run time, which properties are not copied automatically to the new elements?
 - a. Visible, Index, Caption
 - b. Index, Name, Caption
 - c. Visible, Index, TabIndex
 - d. TabIndex, Text, Index
11. Each element in a control array is identified by:
 - a. TabIndex
 - b. Index
 - c. ListIndex
 - d. Subscript
12. In bubble sort how many extra storage variables are needed to re-arrange the two target numbers.
 - a. 0
 - b. 1
 - c. 2
 - d. 6
13. What is the default lower bound of an array?
 - a. -1
 - b. 0
 - c. 1
 - d. Any value
14. If an array has 10 values what is the upper bound of the array.
 - a. 0
 - b. 1
 - c. 9
 - d. 5

15. If an array has 12 values, what is the upper bound of the array.
a. 0 b. 10 c. 11 d. -1
16. How many elements are in the following array?
Dim Arr(10, 10) As Integer
a. 121 b. 2 c. 10 d. 100
17. How many elements are in the following array?
Dim Arr(5, 5) As Integer
a. 5 b. 6 c. 25 d. 36
18. Which of the following statements allocates an array of a dozen integers?
a. Dim Arr(dozen) As Integer b. Dim Arr(12) As Integer
c. Dim Arr(11) As Integer d. Dim Arr(13) As Integer
19. _____ is a set of multiple controls of the same type with same name.
a. Form b. Textbox c. Control Array d. None
20. An integer array with 10 elements can be defined in VB as:
a. Dim C(10) As Integer b. Dim C[10] As Integer
c. Integer C[10] d. Integer C(10)
21. A two dimensional array can viewed as:
a. rows, columns b. One row c. One column d. None
22. One dimensional array is also known as:
a. Linear array b. Vector array c. 1-D array d. All
23. What is the first element of an array having 4 rows and 3 columns:
a. A(0,0) b. A(1,1) c. A(4,3) d. A(2,3)
24. What is the representation of the third element in array called N:
a. N(2) b. N(3) c. N(1) d. N(4)

Answers

1. c	2. b	3. c	4. b
5. a	6. a	7. b	8. a
9. c	10. c	11. b	12. b
13. b	14. c	15. c	16. a
17. d	18. c	19. c	20. a
21. a	22. d	23. a	24. a

Fill in the Blanks

1. The data values of an array are called _____.
2. A set of related data values is best stored in _____.
3. The first element of a zero-based array has an index value of _____.
4. The array Students(_____) contains 20 elements.
5. Salary(0) refers to the _____ element in an array named _____.
6. The UBound function gives the _____ of an array.

7. The ReDim statement is used to _____ an array.
8. The _____ property gives the number of elements in an array.
9. The index value of the last element of an array is equal to the _____.

Answers

1. Elements	2. Array	3. Zero
4. 19	5. First, Salary	6. Upper Boundary
7. Resize	8. Length	9. upper bound

True / False

1. An array consists of multiple elements.
2. The index for the first element of an array is either 0 or 1.
3. Index holds the data values of an array.
4. Arrays always store exactly 10 elements.
5. The statement `Dim intGrades(1 To 10) As Integer` reserves memory for exactly 10 elements.
6. Another name for an array is a list.
7. The default value of the elements in an array is set to 0.
8. The statement `Dim ArrayExample(15) As String` declares an array of integers.
9. Subscript is the term for the position of an element within an array.
10. Use ReDim keyword to change the size of a static array.
11. The upper bound of an array equals the length of the array.
12. The upper bound of an array must be included for declaring an array.
13. Use a dynamic array when the size is unknown at design time.
14. In order to perform a sequential search, the array must first be sorted.
15. In order to perform a binary search, the array must first be sorted.
16. Keep keyword is used to keep previous array values while re-dimensioning an array.
17. The controls in a control array do not share events.
18. Enumerations are used to associate meaningful names with a sequence of constant values.
19. Arrays of a user-defined type can store multiple data types in one array element.
20. Dynamic arrays enhance program performance as they can be resized at execution time.

Answers

1. T	2. F	3. F	4. F
5. T	6. T	7. T	8. F
9. F	10. T	11. F	12. F
13. F	14. T	15. F	16. F
17. T	18. T	19. F	20. F
21. T	22. F	23. F	24. T
25. F	26. F	27. F	

Temperatures of the Week

Project Specification

The project **Temperatures of the Week** displays a form that uses textboxes to input temperatures of a week. An array of textboxes is used to input temperatures of Morning, Noon and Evening for each day. It displays two buttons. One button is used to display average temperature of a given day and second button is used to display maximum temperature of a given day.

Procedure

1. Start a new Standard EXE project.
2. Place the following controls on the form with the given properties:

Control	Control Name	Property	Value
Label	lblTitle	Caption	Temperature of the Week
Label	lblMorning	Caption	Morning
Label	lblNoon	Caption	Noon
Label	lblEvening	Caption	Evening
Label	lblSat	Caption	Saturday
Label	lblSun	Caption	Sunday
Label	lblMon	Caption	Monday
Label	lblTue	Caption	Tuesday
Label	lblWed	Caption	Wednesday
Label	lblThu	Caption	Thursday
Label	lblFri	Caption	Friday
Textbox	Text1(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text1(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text1(2)	Text	0
		Alignment	1 - Right Justify
Textbox	Text2(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text2(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text2(2)	Text	0
		Alignment	1 - Right Justify
Textbox	Text3(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text3(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text3(2)	Text	0
		Alignment	1 - Right Justify

Textbox	Text4(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text4(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text4(2)	Text	0
		Alignment	1 - Right Justify
Textbox	Text5(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text5(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text5(2)	Text	0
		Alignment	1 - Right Justify
Textbox	Text6(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text6(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text6(2)	Text	0
		Alignment	1 - Right Justify
Textbox	Text7(0)	Text	0
		Alignment	1 - Right Justify
Textbox	Text7(1)	Text	0
		Alignment	1 - Right Justify
Textbox	Text7(2)	Text	0
		Alignment	1 - Right Justify
CommandButton	cmdDayMax	Caption	Day Maximum
CommandButton	cmdDayAvg	Caption	Day Average

Note: The project uses seven control arrays of buttons. Each control array represent a day of the week. For example, Text1 control array represents Saturday, Text2 control array represents Sunday and so on.

3. Design the form as follows:

The screenshot shows a Visual Basic form titled "Temperatures of the Week". The form has a table with 5 columns: Day of the Week, Morning, Noon, and Evening. The rows are labeled Saturday, Sunday, Monday, Tuesday, Wednesday, Thursday, and Friday. Each cell in the table contains a text box with the value "0". To the right of the table are two buttons labeled "Day Maximum" and "Day Average".

Day of the Week	Morning	Noon	Evening
Saturday	0	0	0
Sunday	0	0	0
Monday	0	0	0
Tuesday	0	0	0
Wednesday	0	0	0
Thursday	0	0	0
Friday	0	0	0

Day Maximum
Day Average

4. Add the following code in Click event of Day Maximum button:

```

Project1 - Form1 (Code)
cmdDayMax Click
Private Sub cmdDayMax_Click()
    Dim day As Integer, max As Integer
    day = InputBox("Enter day number (1-7)")
    Select Case day
        Case 1
            max = Text1(0)
            For i = 0 To 2
                If max < Text1(i) Then max = Text1(i)
            Next
        Case 2
            max = Text2(0)
            For i = 0 To 2
                If max < Text2(i) Then max = Text2(i)
            Next
        Case 3
            max = Text3(0)
            For i = 0 To 2
                If max < Text3(i) Then max = Text3(i)
            Next
        Case 4
            max = Text4(0)
            For i = 0 To 2
                If max < Text4(i) Then max = Text4(i)
            Next
        Case 5
            max = Text5(0)
            For i = 0 To 2
                If max < Text5(i) Then max = Text5(i)
            Next
        Case 6
            max = Text6(0)
            For i = 0 To 2
                If max < Text6(i) Then max = Text6(i)
            Next
        Case 7
            max = Text7(0)
            For i = 0 To 2
                If max < Text7(i) Then max = Text7(i)
            Next
    End Select
    MsgBox ("Maximum temperature is " & max)
End Sub

```

5. Add the following code in Click event of Day Average control array:

```

Project1 - Form1 (Code)
cmdDayAvg Click
Private Sub cmdDayAvg_Click()
    Dim day As Integer, sum As Integer
    Dim avg As Single
    day = InputBox("Enter day number (1-7)")
    Select Case day
        Case 1
            For i = 0 To 2
                sum = sum + Text1(i)
            Next
        Case 2
            For i = 0 To 2
                sum = sum + Text2(i)
            Next
        Case 3
            For i = 0 To 2
                sum = sum + Text3(i)
            Next
        Case 4
            For i = 0 To 2
                sum = sum + Text4(i)
            Next
    End Select
    avg = sum / 3
    MsgBox ("Average temperature is " & avg)
End Sub

```

```

Case 5
For i = 0 To 2
    sum = sum + Text5(1)
Next
Case 6
For i = 0 To 2
    sum = sum + Text6(1)
Next
Case 7
For i = 0 To 2
    sum = sum + Text7(1)
Next
End Select
avg = sum / 3
MsgBox ("Average temperature is " & avg)
End Sub

```

6. Run the project and enter values as follows:

	Morning	Noon	Evening	
Saturday	10	22	18	
Sunday	12	25	16	
Monday	15	23	14	
Tuesday	12	26	15	
Wednesday	11	27	13	
Thursday	14	24	20	Day Maximum
Friday	11	25	22	Day Average

7. Click Day Maximum button. An input box will appear.
 8. Enter 2 for Sunday as follows:

Project1

Enter day number (1-7)

2

OK Cancel

9. Click OK. The result will appear as follows:

Project1

Maximum temperature is 25

OK

10. Click Day Average button. An input box will appear.
 11. Enter 1 for Saturday.
 12. Click OK. The result will appear as follows:

Project1

Average temperature is 16.66667

OK

4. Run the project. The output will appear as follows:

Number	Cube
1	1
2	8
3	27
4	64
5	125

Program 6.9

Write a program that inputs a number. It then displays the sum of all odd numbers and the sum of all even numbers from 1 to the number entered by the user. For example, if the user enters 5, the sum of odd numbers is 9 and sum of even numbers is 6.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.

```

Private Sub Command1_Click()
    Dim n As Integer
    Dim oddsum As Integer
    Dim evensum As Integer
    oddsum = 0
    evensum = 0
    n = Val(InputBox("Enter a number to calculate sum:"))
    Do While n >= 0
        If n Mod 2 = 0 Then
            evensum = evensum + n
        Else
            oddsum = oddsum + n
        End If
        n = n - 1
    Loop
    Print "The sum of even digits is:" & evensum
    Print "The sum of odd digits is:" & oddsum
End Sub

```

4. Run the project and test the working.

Program 6.10

Write a program that inputs a number and check whether it is an Armstrong number or not. A number is an Armstrong number if the sum of the cubes of its digits is equal to the number itself. For example, 371 is an Armstrong number since $3^3 + 7^3 + 1^3 = 371$.

1. Start a new Standard EXE project.
2. Place a button on the form.
3. Double click the button and add the following code in its Click event.